

## Hide Delegate - After

### Why Refactor

:To start with, let's look at terminology

.Server is the object to which the client has direct access

Delegate is the end object that contains the functionality needed by the  
.client

A call chain appears when a client requests an object from another object, then the second object requests another one, and so on. These sequences of calls involve the client in navigation along the class structure. Any changes in these interrelationships will require changes  
.on the client side

### Benefits

Hides delegation from the client. The less that the client code needs to know about the details of relationships between objects, the easier it's to  
.make changes to your program

### Drawbacks

If you need to create an excessive number of delegating methods, server-class risks becoming an unneeded go-between, leading to an  
.excess of Middle Man

### How to Refactor

For each method of the delegate-class called by the client, create a  
.method in the server-class that delegates the call to the delegate-class

.Change the client code so that it calls the methods of the server-class

If your changes free the client from needing the delegate-class, you can remove the access method to the delegate-class from the server-class  
(the method that was originally used to get the delegate-class)

## Hide Delegate 🐒

### مشکل (Problem)

کلاينت از فيلد يا متد شيء A، شيء B رو می گیره. بعد کلاينت يه متد از شيء B رو صدا میزنه.

## راه حل (Solution)

یه متد جدید در کلاس A بساز که صدا زدن رو به شیء B تفویض (Delegate) کنه. حالا کلاینت دیگه از کلاس B خبر نداره و بهش وابسته نیست.

### مثال از صفحه

قبل (کلاینت B رو مستقیم صدا میزنه):

csharp

```
Client //
;()var manager = department.GetManager
;Console.WriteLine(manager.GetName())
```

بعد (مخفی کردن Delegate):

csharp

```
Client //
;Console.WriteLine(department.GetManagerName())

Department //
()public string GetManagerName
}

Manager به تفویض // ;()return manager.GetName
{
```

## چرا ریفکتور کنیم؟ (Why Refactor)

مفهوم

توضیح

Server

شیئی که کلاینت مستقیماً بهش دسترسی داره (Department)

Delegate

شیء نهایی که عملکرد مورد نیاز کلاینت رو داره (Manager)

## مفهوم

## توضیح

وقتی کلاینت از یه شیء، شیء دیگه‌ای رو می‌گیره و ادامه میده... این یعنی کلاینت در ساختار کلاس‌ها پیمایش می‌کنه. هر تغییری در این روابط، کلاینت رو مجبور به تغییر می‌کنه

Call chain

 مزایا (Benefits)

پنهان کردن delegation از کلاینت. هرچی کلاینت کمتر از جزئیات روابط بین اشیاء بدونه، تغییرات برنامه آسون‌تر میشه.

 نقطه ضعف (Drawbacks)

اگه تعداد زیادی متد تفویضی بسازی، Server-class تبدیل به یه **Middle Man** بی‌خاصیت میشه.

 How to Refactor (چطور انجام بدیم)

## قدم

## توضیح

۱ برای هر متد از Delegate که کلاینت صدا میزنه، یه متد در **Server** بساز که به Delegate تفویض کنه

۲ کد کلاینت رو طوری تغییر بده که متدهای **Server** رو صدا بزنه

۳ اگه کلاینت دیگه به Delegate نیاز نداره، متد دسترسی به Delegate رو از **Server** حذف کن

 مثال ساده

قبل:

csharp

```

Client //
;()var person = new Person
;()var department = person.GetDepartment
;()var manager = department.GetManager
;Console.WriteLine(manager.GetName())

```

بعد:

csharp

```

Client //
;()var person = new Person
;Console.WriteLine(person.GetManagerName())

Person //
()public string GetManagerName
}
;()return department.GetManager().GetName
{

```

تمرین 

این کد رو با Hide Delegate اصلاح کن:

csharp

```

public class Order
}

public Customer Customer { get; set; }
{

public class Customer
}

public Address Address { get; set; }
{

public class Address
}

public string City { get; set; }
public string Street { get; set; }

```

```

{

:Client //
;()var order = new Order
;var city = order.Customer.Address.City // زنجیره دراز!

```

راهنمایی: متد GetDeliveryCity به Order اضافه کن.

بفرست تا بررسی کنیم! 🙌

خودت حلش کن

## کد اصلاح شده با Hide Delegate

```

csharp

public class Order
{
    public Customer Customer { get; private set; }

    ()public string GetDeliveryCity
    {
        ;"نامشخص" ?? ()return Customer?.GetCity
    }
}

public class Customer
{
    public Address Address { get; set; }

    ()public string GetCity
    {
        ;"نامشخص" ?? return Address?.City
    }
}

public class Address
{
    public string City { get; set; }
    public string Street { get; set; }
}

```

```

:Client //
;()var order = new Order
var city = order.GetDeliveryCity // ساده و تمیز!

```

## مقایسه

| نسخه اصلاح                                                                                                | کد اصلی                                                                                                       |                      |
|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|----------------------|
|  ()order.GetDeliveryCity |  order.Customer.Address.City | کلاینت               |
|  فقط به Order            | به Order + Customer + Address                                                                                 | وابستگی کلاینت       |
|  فقط Order و Customer    | کلاینت باید تغییر کنه                                                                                         | تغییر ساختار Address |
|  ۱ مرحله                 | ۳ مرحله                                                                                                       | پیمایش زنجیره        |

## Hide Delegate تموم شد!

و با این، **Inappropriate Intimacy** کامل شد. بریم سراغ **Message Chains**؟ متنش رو بفرست! 